

▲ Apache Airflow

ORCHESTRATION

PYTHON >= 3.8

Orchestration de pipelines de données — Fiche de référence rapide

Contexte TP : Airflow orchestre votre ETL imprimantes — il planifie l'extraction MySQL/TXT/Excel/JSON, déclenche les transformations et charge les données en PostgreSQL, le tout de façon automatisée et monitored.

CONCEPTS FONDAMENTAUX

VOCABULAIRE ESSENTIEL

DAG	Directed Acyclic Graph — le pipeline complet, définit l'ordre des tâches
Task	Unité de travail atomique (un Operator instancié)
Operator	Template de tâche : PythonOperator, BashOperator, PostgresOperator...
DAG Run	Exécution concrète d'un DAG à un instant donné
Scheduler	Service qui déclenche les DAGs selon leur schedule
XCom	Mécanisme pour passer des données entre tâches
Connection	Credentials stockés (DB, API) — référencés par conn_id

OPÉRATEURS LES PLUS UTILISÉS

PythonOp.	Exécute une fonction Python — le plus flexible
BashOp.	Exécute une commande shell
PostgresOp.	Exécute un SQL sur PostgreSQL via conn_id
EmailOp.	Envoie un email en cas d'alerte
BranchOp.	Branchement conditionnel du flux
DummyOp.	Tâche vide — pour structurer le DAG
FileSensor	Attend l'apparition d'un fichier avant de continuer

STRUCTURE D'UN DAG — TP IMPRIMANTES

DAG_ETL_IMPRIMANTES.PY

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.postgres import PostgresOperator
from airflow.sensors.filesystem import FileSensor
```

```

from datetime import datetime, timedelta

# — 1. PARAMÈTRES PAR DÉFAUT —————
default_args = {
    'owner': 'data_team',
    'retries': 2,                                # 2 tentatives en cas d'échec
    'retry_delay': timedelta(minutes=5),
    'email_on_failure': True,
    'email': ['data@entreprise.fr'],
}

# — 2. DÉCLARATION DU DAG —————
with DAG(
    dag_id='etl_imprimantes',
    default_args=default_args,
    schedule_interval='0 6 * * 1-5',           # Tous les jours ouvrés à 6h
    start_date=datetime(2024, 1, 1),
    catchup=False,                             # Pas de backfill automatique
    tags=['dwh', 'imprimantes'],
) as dag:

    # — 3. TÂCHES —————

    attendre_fichier = FileSensor(
        task_id='attendre_fichier_clients',
        filepath='/data/sources/02_clients.txt',
        poke_interval=60,                      # Vérifie toutes les 60s
    )

    extraire_mysql = PythonOperator(
        task_id='extraire_mysql',
        python_callable=extract_mysql_sql,     # Fonction de votre ETL
    )

    extraire_sources = PythonOperator(
        task_id='extraire_sources',
        python_callable=extract_all_sources,
    )

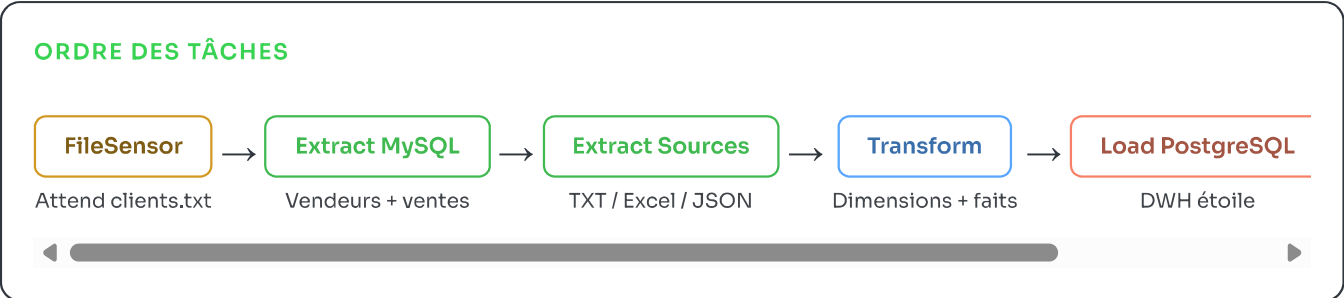
    transformer = PythonOperator(
        task_id='transformer_dimensions',
        python_callable=run_transformations,
    )

    charger_pg = PostgresOperator(
        task_id='charger_postgres',
        postgres_conn_id='postgres_dwh',      # Défini dans Connections UI
        sql='sql/load_faits.sql',
    )

    # — 4. DÉPENDANCES (ordre d'exécution) —————
    attendre_fichier >> extraire_sources
    extraire_mysql >> extraire_sources
    extraire_sources >> transformer >> charger_pg

```

FLUX D'EXÉCUTION DU DAG



STATUTS DES TÂCHES

success

failed

running

queued

skipped

upstream_failed

no_status

Attention :

upstream_failed

— la tâche n'a pas échoué elle-même, mais une tâche amont a échoué.

COMMANDES CLI ESSENTIELLES

<code>airflow db init</code>	Initialise la base de métadonnées
<code>airflow webserver -p 8080</code>	Démarre l'UI web
<code>airflow scheduler</code>	Démarre le planificateur
<code>airflow dags list</code>	Liste tous les DAGs
<code>airflow dags trigger etl_imp.</code>	Déclenche manuellement
<code>airflow tasks test etl_imp. extraire_mysql 2024-01-01</code>	Teste une tâche seule
<code>airflow dags backfill -s 2024-01-01 -e 2024-01-31 etl_imp.</code>	Rejoue sur période

SCHEDULE — NOTATION CRON

'@daily'

Chaque jour à minuit

'@weekly'

Chaque dimanche

'@hourly'

Chaque heure

'0 6 * * 1-5'

Lun-Ven à 6h00

'0 22 * * *'

Tous les jours à 22h

'*/15 * * * *'

Toutes les 15 minutes

None

Déclenche uniquement manuellement

BONNES PRATIQUES

Idempotence

Un DAG re-exécuté doit donner le même résultat (INSERT OR REPLACE)

Atomicité

Chaque tâche doit réussir ou échouer entièrement, sans état partiel

catchup=False

Évite le backfill automatique lors du 1er démarrage

Connections UI

Ne jamais hardcoder les credentials dans le code

[Logs](#)

Toujours logger dans les fonctions Python pour traçabilité

INSTALLATION RAPIDE (LOCAL)

SETUP EN 4 COMMANDES

1. Installation

```
pip install apache-airflow==2.8.0 --constraint "https://raw.githubusercontent.com/apache/airflow
```

2. Initialisation

```
export AIRFLOW_HOME=~/.airflow
airflow db init
```

3. Créer un utilisateur admin

```
airflow users create --role Admin --username admin --password admin \
  --firstname Admin --lastname User --email admin@example.com
```

4. Démarrer (deux terminaux séparés)

```
airflow webserver --port 8080 # → http://localhost:8080
airflow scheduler
```

Astuce : Placez vos fichiers DAG dans `~/airflow/dags/`. Le scheduler les détecte automatiquement (délai ~30s).